Functions

1. Introduction

- *Breaking a complex program into smaller parts is a common practice*. We divide programs into modules that are tractable.
- *A **function** is an entity designed to do a task; it has a **header** and **a set of statements** enclosed in opening and closing braces*.
- *Previously, we used a single function, main, to do the whole task of the program*. In other words, we put the whole responsibility of solving a problem on the *main* function. *This approach works for small programs in which the task is very simple and the program is short*.

1. Introduction (continued…)

- When a task is more complicated, we divide the task into smaller tasks, in which each task is responsible for a part of the job. For example, manufacturing a car is divided into several tasks. The body is made at one location, the engine is made somewhere else, another entity is responsible for making tires, and so on. When all of the parts are made and ready, the car is assembled. The same approach is applied to programming.

1. Introduction (continued…)

- Figure given below shows this approach.

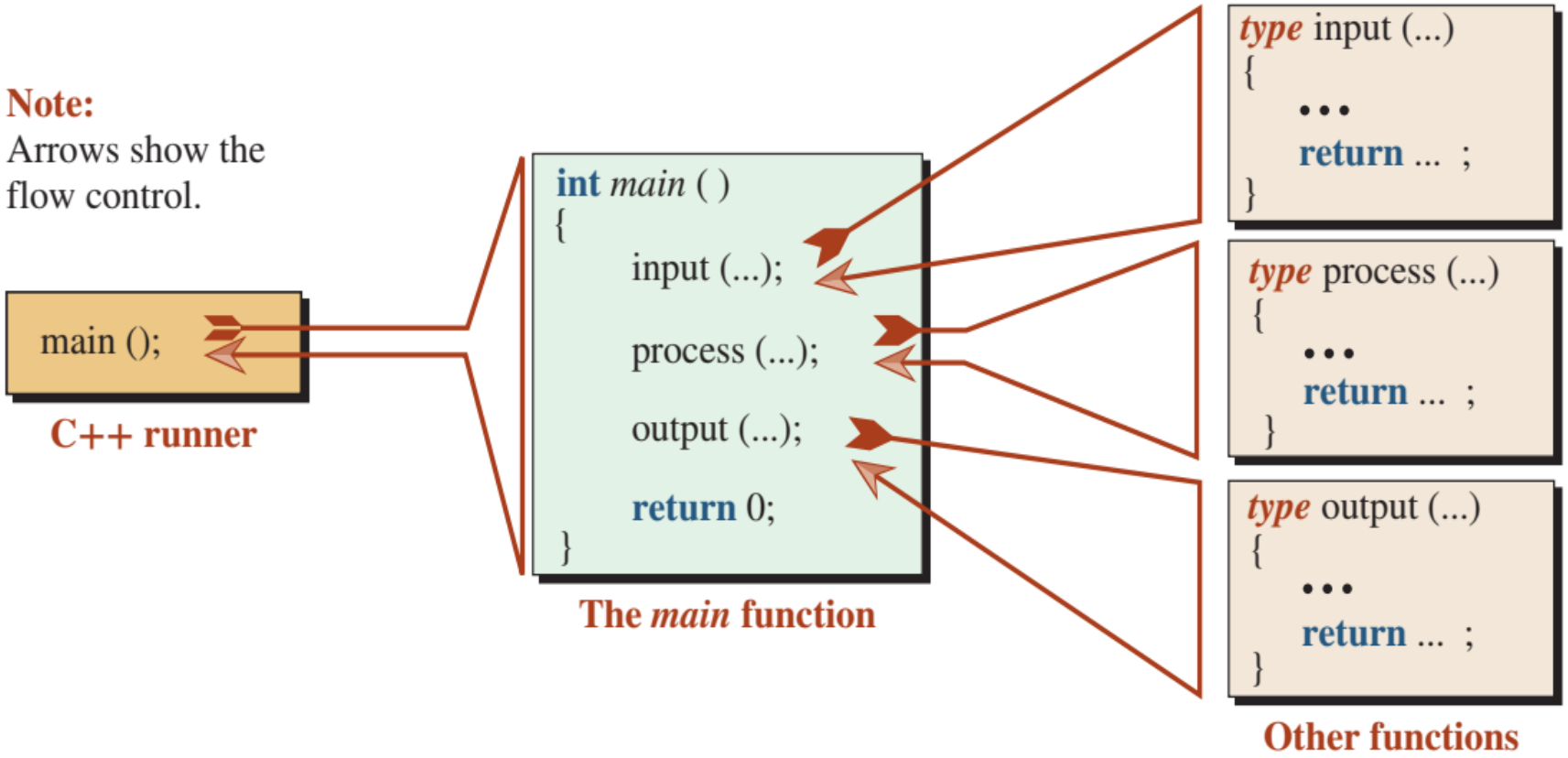


Figure 1: A program made of several functions

1.1 Benefits

i. Easier to Write Simpler Task

Everyone knows that doing simple tasks is easier than doing difficult tasks. It is easier to concentrate on manufacturing tires for cars than manufacturing the whole car.

ii. Error Checking (Debugging)

One of the big headaches of programming is finding errors (called bugs). Error checking (or debugging) is much more simple when a program is divided into small functions. Each function can be debugged and then put together as one program.

1.1 Benefits (continued…)

iii. Reusability

Small tasks can be reused in many large tasks. If we isolate these small tasks and write a function for each, we can create many large programs by assembling these small functions. We do not need to rewrite the small task over and over again.

1.2 Definition, Declaration and Call

To work with function, we must consider three entities: *function definition, function declaration, and function call*. We briefly discuss these here:

Function Definition

The **function definition** *creates the function*. Like any other entity in C++, the definition of a function must follow syntax rules. Figure 2 below shows the basic syntax of a function definition.

```
return-type function-name (parameter list)
{
        Body
}
```

Figure 2: Syntax of function definition

1.2 Definition, Declaration and Call (continued…)

As the figure in previous slide shows, the definition is made of two sections: the **function header** and the **function body**.

Figure 3 below shows the definition of the function **abs** in more detail. It identifies various parts of this function.

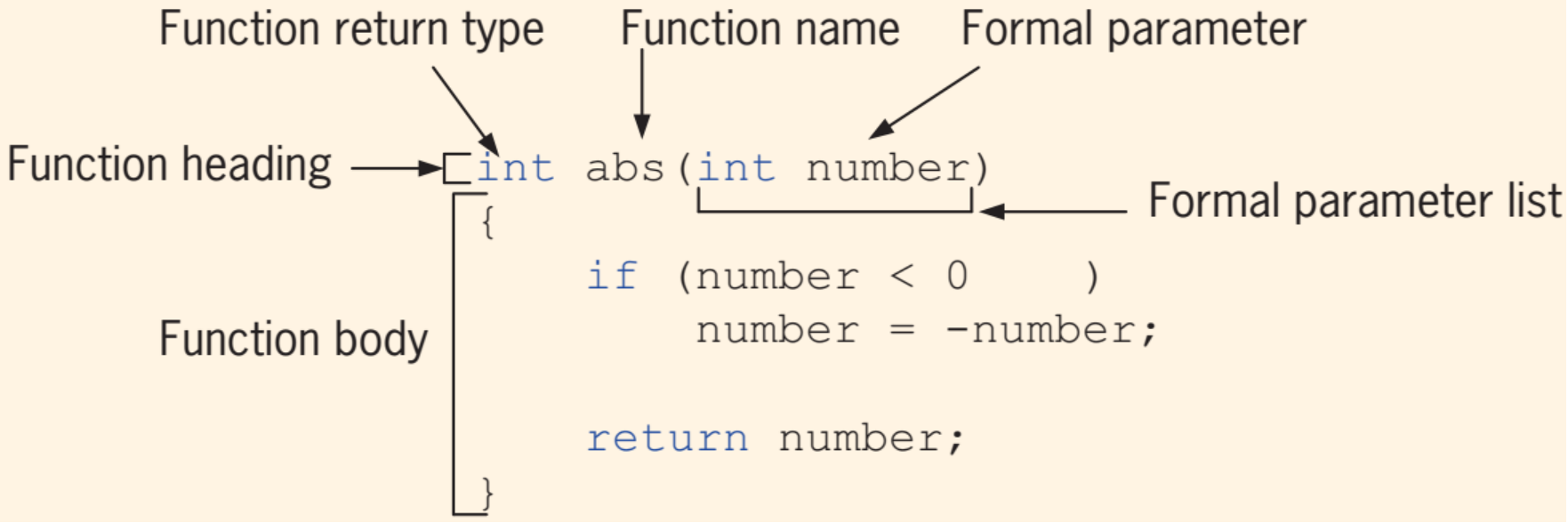


Figure 3: Various parts of the function abs

1.2 Definition, Declaration and Call (continued…)

Function Declaration

The **function declaration** (also called **function prototype**) *is only the header of the function followed by a semicolon*. The parameter names are optional; the types are required. *The function declaration is used to show how the function should be called.* Example given below shows the declaration of the function defined in figure 3:

- int abs (int number); // with the names of the parameters
- int abs (int); // without the names of the parameters

1.2 Definition, Declaration and Call (continued…)

Function Call

*The **function calling** is actually executing the statements of a function to perform a task. The parameters (if any) of the function are given in parentheses after the name of the function. If a function has no parameters then the parentheses are left blank.*

When a function is called, the control shifts to the function definition and the statements in the body of the function are executed. After executing the statements in the body of the function, the control returns to the calling function and the next statement that comes immediately after the function call statement is executed.

1.2 Definition, Declaration and Call (continued…)

In the following program given below, function “sum” adds two integer numbers. The numbers whose sum is to be calculated are passed to the function as arguments.

```
int main()
{
        void sum (int x, int y);
        sum (10,15);
        return 0;
}

void sum (int x, int y)
{
        int s;
        s = x + y;
        cout<<"Sum = "<<s<<endl;
}
```

Function Declaration

Function Calling

Function Definition

Output

```
Sum =; 25

--------------------------------
Process exited after 0.05667 seconds with return value 0
Press any key to continue . . .
```

1.2 Definition, Declaration and Call (continued…)

Arguments and Parameters

- A *parameter* is a variable declared in the header of the function definition.
- An *argument* is a value that initializes the parameter when the function is called.

```
int main ()
{
   …
   fun (5);              // Function call; 5 is an argument
   …
}
void fun (int x)         // Function definition; x is a parameter
{
   …
}
```

x = 5

1.3 Library and User-Defined Functions

To use functions in a program, we must have a definition and a function call. However, we must choose between two types: *library functions* and *user-defined functions*.

Library Functions

The C++ library contains predefined functions. We need only the declaration of the functions to call them. *We will discuss only the commonly used library functions.*

User-Defined Functions

There are a lot of functions that we need, but there are no predefined functions for them. *We must define these functions first and then call them.*

2. Library Functions

As we said before, if there is a predefined function, we do not need to worry about the function definition. *We only add the corresponding header file in which the function is defined*. However, we must call the function, which means that we must know the declaration of the function. *Some of the commonly used library functions are discussed in next slide(s)*.

2.1 Mathematical Functions

C++ defines a set of mathematical functions that can be called in our program. These functions are predefined and collected under the *<cmath>* header.

Numeric Functions

- Table 1 next slide shows the return values of numeric functions.
- Numeric functions are used in numeric calculations.
- The type is *int*, *float*, *double*, or *long double*.
- In the case of the *pow* function, the second argument can also be an integer.

2.1 Mathematical Functions (continued…)

- Program next slide shows the return values of the functions given in table 1

Some numeric functions defined in <cmath>	
Function declaration	**Explanation**
type abs (type *x*);	Returns absolute value of *x*
type ceil (type *x*);	Returns largest integral value less than or equal to *x*
type floor (type *x*);	Returns smallest integral value less than or equal to *x*
type log (type *x*);	Returns the natural (base e) logarithm of *x*
type log10 (type *x*);	Returns the common (base 10) logarithm of *x*
type exp (type *x*);	Returns e^x
type pow (type *x*, type *y*);	Returns x^y; note that y can also be of type int in this case
type sqrt (type *x*);	Returns the square root of *x* ($x^{1/2}$)

Table 1

2.1 Mathematical Functions (continued…)

```
1  /*************************************************************
2    * This program shows how to use some of the numeric functions      *
3    * defined in the <cmath> header file.                              *
4    *************************************************************/
5  #include <iostream>
6  #include <cmath>
7  using namespace std;
8
9  int main ( )
10 {
11    // abs function with two different arguments
12    cout << "abs (8) = " << abs (8) << endl;
13    cout << "abs (-8) = " << abs (-8) << endl;
14    // floor and ceil functions with the same argument
15    cout << "floor (12.78) = " << floor (12.78) << endl;
16    cout << "ceil (12.78) = " << ceil (12.78) << endl;
```

2.1 Mathematical Functions (continued…)

```
17    // log and log10 functions
18    cout << "log (100) =  " << log (100) << endl;
19    cout << "log10 (100) =  " << log10 (100) << endl;
20    // exp and pow functions
21    cout << "exp (5) =  " << exp (5) << endl;
22    cout << "pow (2, 3) = " << pow (2,3) << endl;
23    // sqrt function
24    cout << "sqrt (100)     " << sqrt (100);
25    return 0;
26 }
```

```
Run:
abs(8) = 8
abs(−8) = 8
floor(12.78) = 12
ceil(12.78) = 13
log(100) = 4.60517
log10 (100) = 2
exp(5) = 148.413
pow(2, 3) = 8
sqrt(100) = 10
```

2.1 Mathematical Functions (continued…)

Home Work Assignment

PROGRAM 1: For an application of one of these functions, *pow*, find the roots of the quadratic equation: $ax^2+bx+c=0$ as we have seen in mathematics. We know that three situations can happen depending on the value of (b^2-4ac), which we call the term:

a. If the value of the *term* is negative, there are no real roots (they are complex).
b. If the value of the *term* is zero, the two roots are the same and the common value is $-b/2a$.
c. If the value of the *term* is positive, the two roots are distinct and their values are $-b + (b^2-4ac)^{1/2}$ and $-b - (b^2-4ac)^{1/2}$.

Write a program considering above three facts to find the roots of any quadratic expression *using mathematical function(s) defined in Table 1* .

2.1 Mathematical Functions (continued…)

Trigonometric Functions

- Table 2 given below shows the list of common trigonometric functions.
- Note that the argument of the first three functions needs to be in radians, in which 180 degrees = π radians and (π = 3.14) approximately.

Trigonometric functions			
Function declaration	**Function explanation**	**Argument units**	**Returned range**
type cos (type *x*);	Returns the cosine	radians	[−1, +1]
type sin (type *x*);	Returns the sine	radians	[−1, +1]
type tan (type *x*);	Returns the tangent	radians	any
type acos (type *x*);	Returns the inverse cosine	[−1, +1]	[0, π]
type asin (type *x*);	Returns the inverse sine	[−1, +1]	[0, π]
type atan (type *x*);	Returns the inverse tangent	any	[−π/2, +π/2]

Table 2

2.1 Mathematical Functions (continued…)

```
1  /*************************************************
2    * A program to test some trigonometric functions
3    *************************************************/
4  #include <iostream>
5  #include <cmath>
6  using namespace std;
7
8  int main ( )
9  {
10   // Defining constant PI and an angle of 45 degrees
11   const double PI  = 3.14159265358979238462;
12   double degree = PI / 4;
13   // Finding the sin, cos, and tan of an angle of 45 degrees
14   cout << "sin (45): " << sin (degree) << endl;
15   cout << "cos (45): " << cos (degree) << endl;
16   cout << "tan (45): " << tan (degree);
17   return 0;
18 }
```

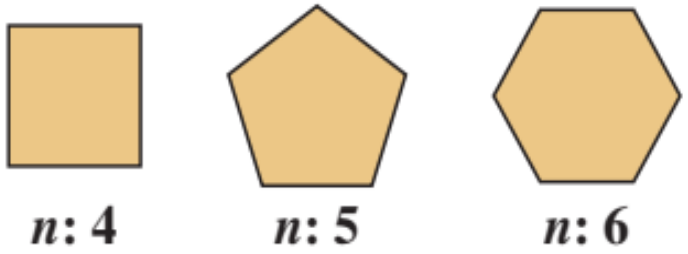
```
Run:
sin (45): 0.707107
cos (45): 0.707107
tan (45): 1
```

2.1 Mathematical Functions (continued…)

Home Work Assignment

PROGRAM 2: Trigonometry tells us that we can find the perimeter and area of a polygon with four or more sides given the number of sides and the length of each side as shown in figure given below:

Write a program to calculate the perimeter and the area of any polygon. Also show its output.



$$\text{perimeter} = n \times s$$

$$\text{area} = \frac{n \times s^2}{4 \times \tan(\pi/n)}$$

Figure 4: Perimeter and area of polygons

3. User-Defined Functions

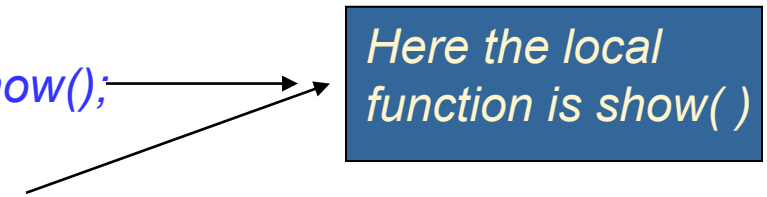
Scope of Functions

The area in which a function can be accessed is known as the scope of a function. The scope of any function is determined by the place of function declaration. Different types of functions on the basis of scope are as follows:

Local Functions

A type of function that is declared within another function is called local function. A local function can be called within the function in which it is declared.

```
int main()
{
 void show();
abs();
show();
Return 0;
}
```



3. User-Defined Functions (continued…)

Global Functions

A type of function that is declared outside any function. A global function can be called from any part of the program.

4. Passing Parameters to Functions

Parameters are the values that are passed to a function when the function is called. Parameters are given in the parentheses. If there are many parameters, these are separated by commas. If there is no parameter, empty parenthesis are used. *Both the **variables** and **constants** can be passed to a function as parameters*.

The sequence and types of parameters in **function call** must be similar to the sequence and types of parameters in **function declaration**.

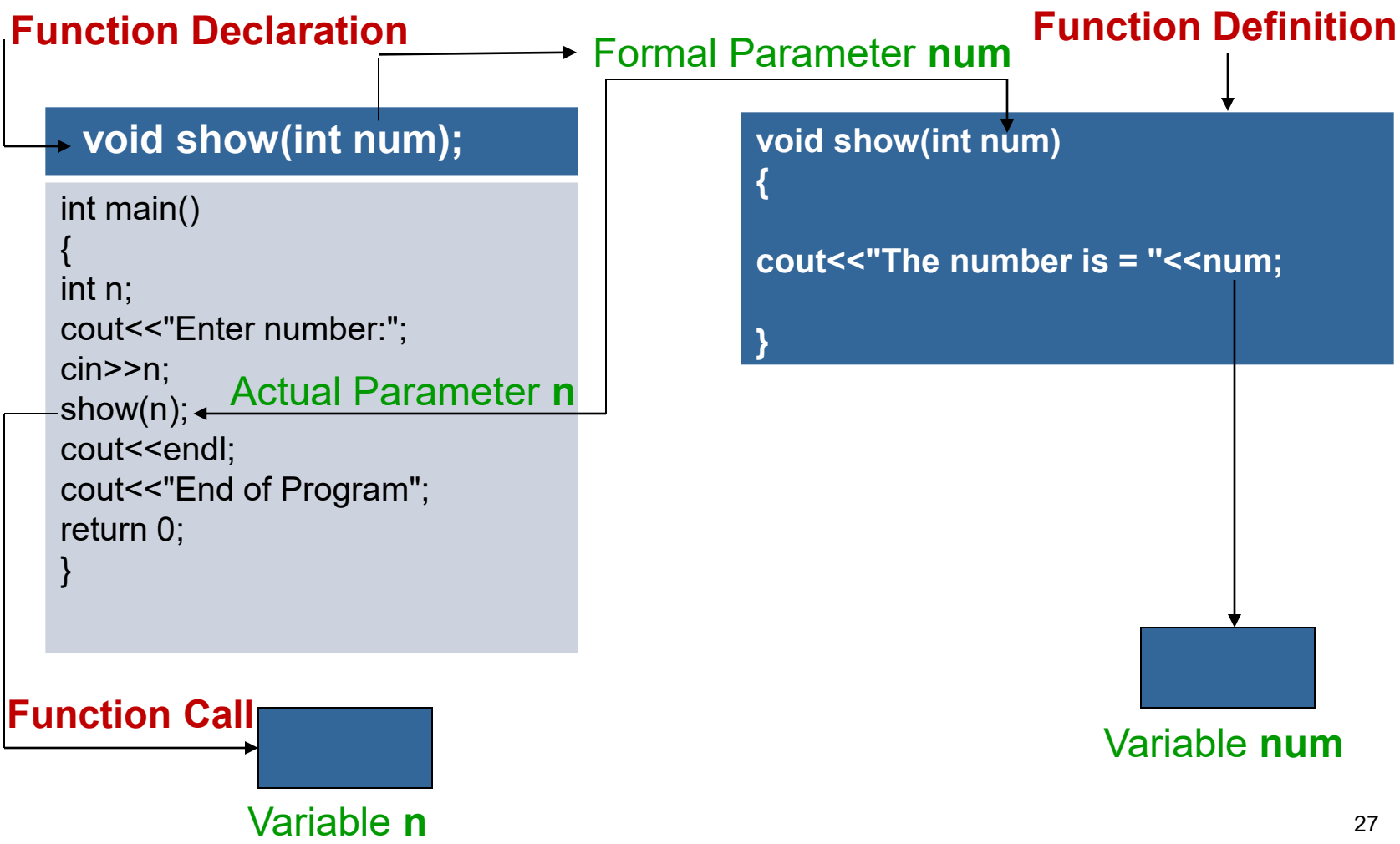
- Parameters in function call are called **actual parameters**.
- Parameters in function declaration are called **formal parameters**.

When a function call is executed, the values of actual parameters are copied to formal parameters.

4.1 Pass by Value

A parameter passing mechanism in which the value of actual parameter is copied to formal parameters of called function is known as ***pass by value***. The process of passing by value is explained with the help of a program in next slide.

4.1 Pass by Value (continued…)

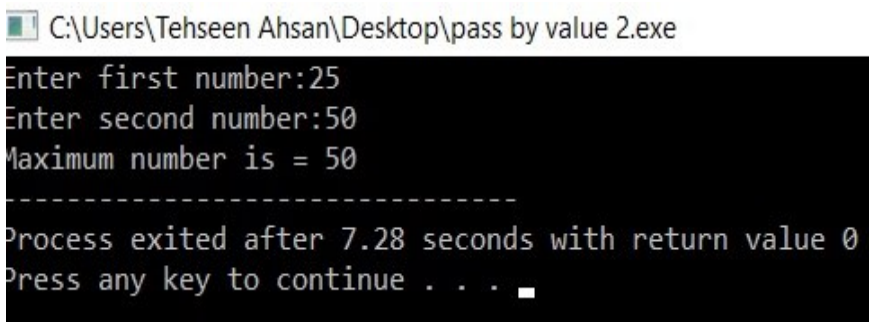


4.1 Pass by Value (continued…)

Example: A program that inputs two numbers in main() function, passes these numbers to a function. The function then displays the maximum number.

```
#include <iostream>
using namespace std;
void max(int a, int b); // Function Declaration
int main()
{
int x, y;
cout<<"Enter first number:";
cin>>x;
cout<<"Enter second number:";
cin>>y;
max(x, y); // Function Call
return 0;
}

void max(int a, int b ) // Function Definition
{
        if(a>b)
        cout<<"Maximum number is = "<<a;
        else
        cout<<"Maximum number is = "<<b;

}
```

Output



4.1 Pass by Value (continued…)

Home Work Assignment

PROGRAM 3: Write a program that inputs a number in main function and passes the number to a function. The function displays table of that number. Also show it's output.

4.2 Returning Value from Function

A function can return a single value. *The return type in function declaration indicates the type of value returned by a function*. For example, **int** is used as return type if the function returns integer value. If the function returns no value, the keyword **void** is used as return type. *The keyword **return** is used to return the value back to the calling function*.

When the **return** statement is executed in a function, the control moves back to the calling function along with the returned value.

Syntax

The syntax for returning a value is as follows:

return expression;

Expression: *It can be a variable, constant or an arithmetic expression whose value is returned to the calling function*. The calling function can use the returned value in the following ways:

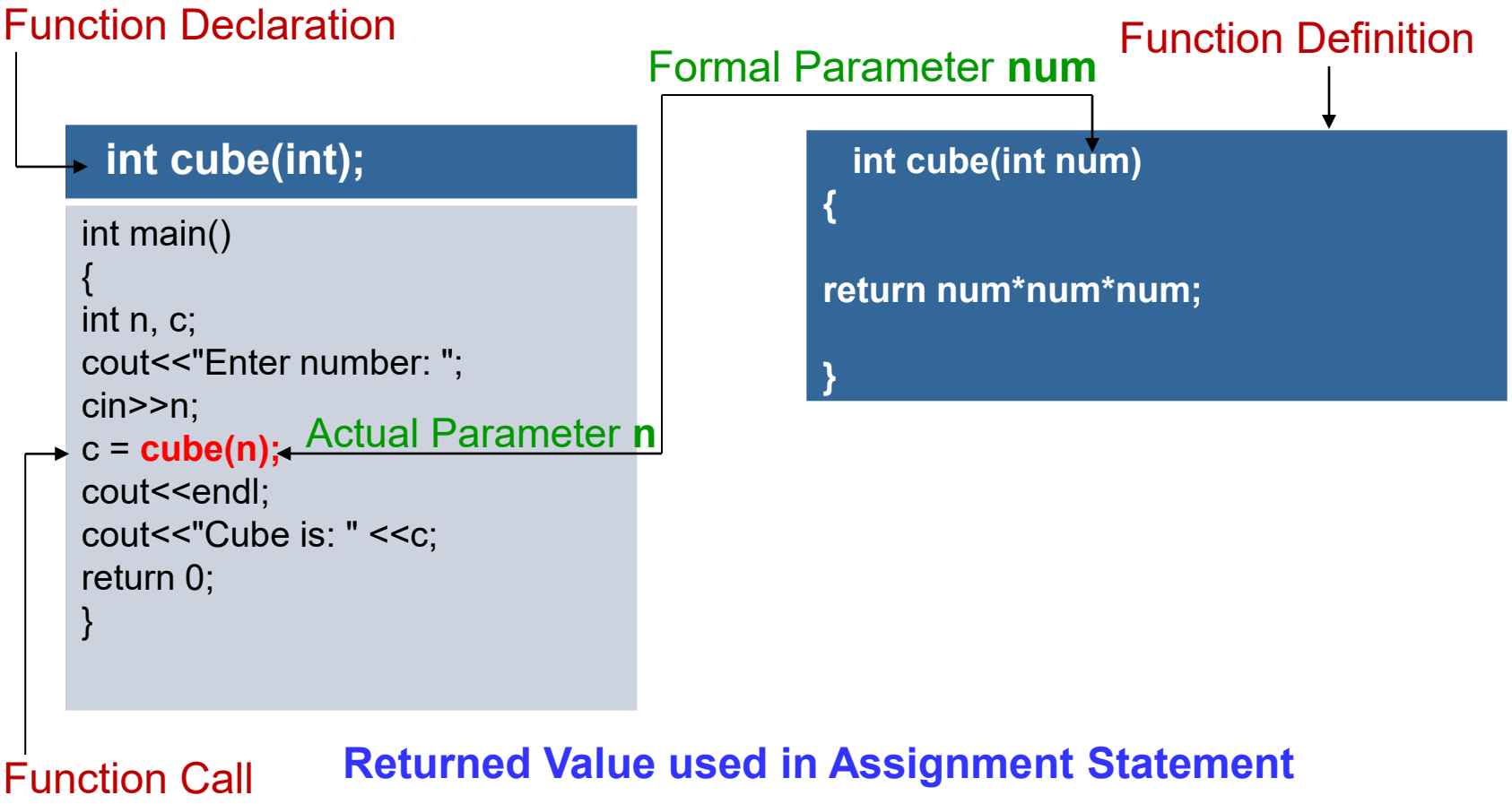
1. ***Assignment statement***
2. ***Arithmetic statement***
3. ***Output statement***

4.2 Returning Value from Function (continued…)

1. Assignment Statement

The calling function can store the returned value in a variable and then use this variable in the program. It is explained with the help of a diagram in next slide.

4.2 Returning Value from Function (continued…)



4.2 Returning Value from Function (continued…)

Example: A program that inputs marks in the main function and passes these marks to a function. The function finds the grade of student on the basis of the following criteria:

Grade A	**80 or Above marks**
Grade B	**60 to 79 marks**
Grade C	**40 to 59 marks**
Grade F	**below 40 marks**

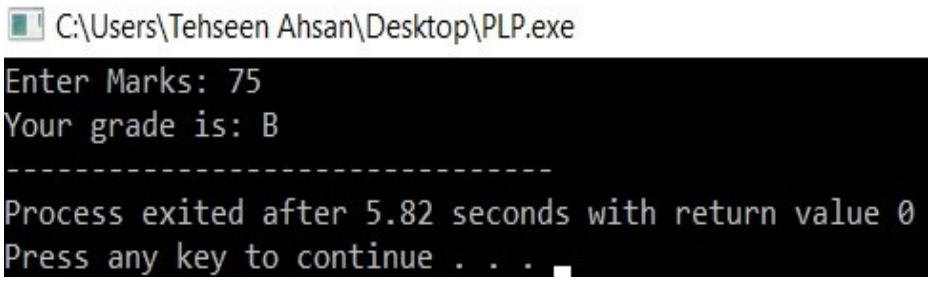
```
#include <iostream>
using namespace std;

char grade(int m);

int main()
{
        int marks;
        char g;
        cout<<"Enter Marks: ";
        cin>>marks;
        g = grade(marks);
        cout<<"Your grade is: "<<g;
        return 0;
}
```

Next slide

4.2 Returning Value from Function (continued…)

```
char grade(int m)
{
        if(m>80)
        return 'A';
        else if(m>60)
        return 'B';
        else if(m>40)
        return 'C';
        else
        return 'F';
}
```

Output



4.2 Returning Value from Function (continued…)

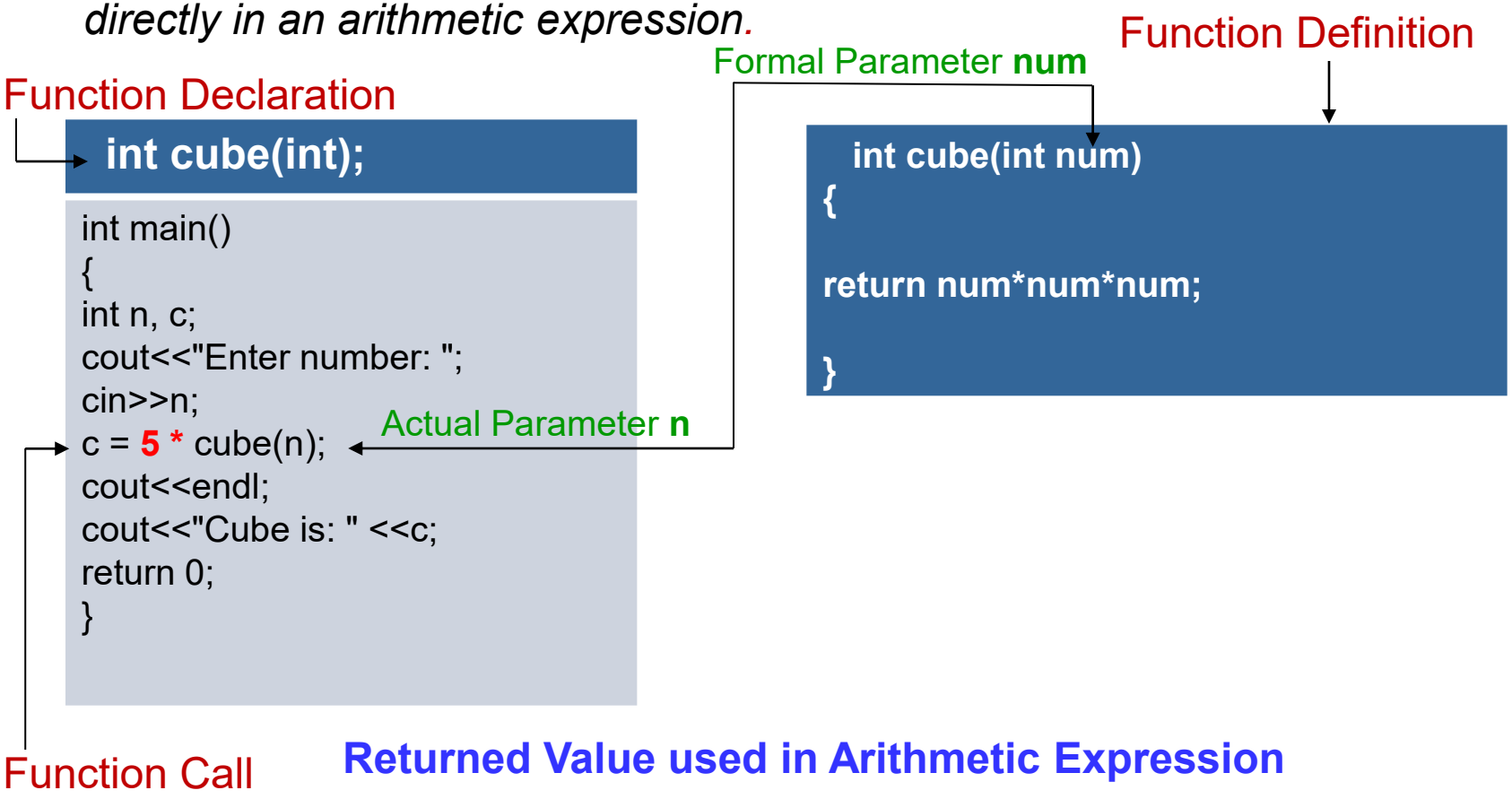
Home Work Assignment

PROGRAM 4: Write a program that inputs base and height of a triangle in main function and passes them to a function. The function finds the area of triangle and returns it to main function where it is displayed on the screen.

*Area = 1 / 2(Base*Height)*

4.2 Returning Value from Function (continued…)

2. **Arithmetic Expression:** *The calling function can use the returned value directly in an arithmetic expression.*



4.2 Returning Value from Function (continued…)

Example: A program that inputs two integers. It passes first integer to a function that calculates and returns its square. It passes second integer to another function that calculates and returns its cube. The **main()** function adds both returned values and displays the result.

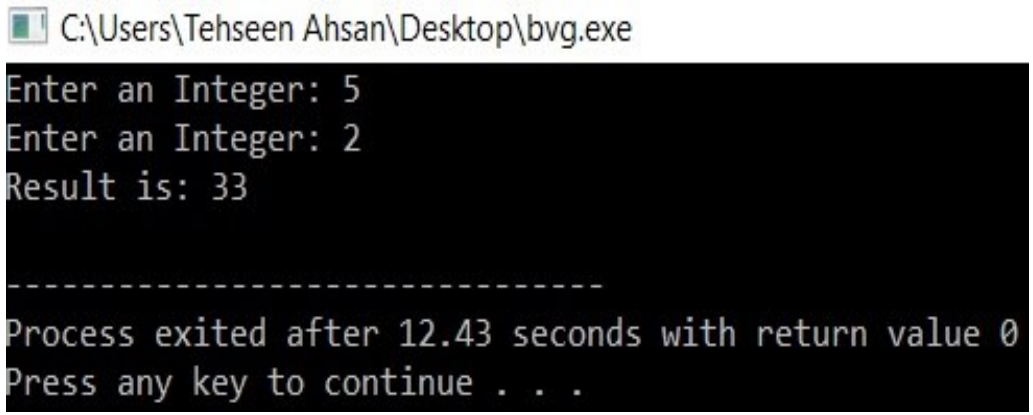
```
#include<iostream>
using namespace std;

int sqr(int n);
int cube(int n);

int main()
{
int a, b, r;
cout<<"Enter an Integer: ";
cin>>a;
cout<<"Enter an Integer: ";
cin>>b;
r = sqr(a) + cube(b);
cout<<"Result is: "<<r<<endl;
return 0;
}
```

Next slide

4.2 Returning Value from Function (continued…)

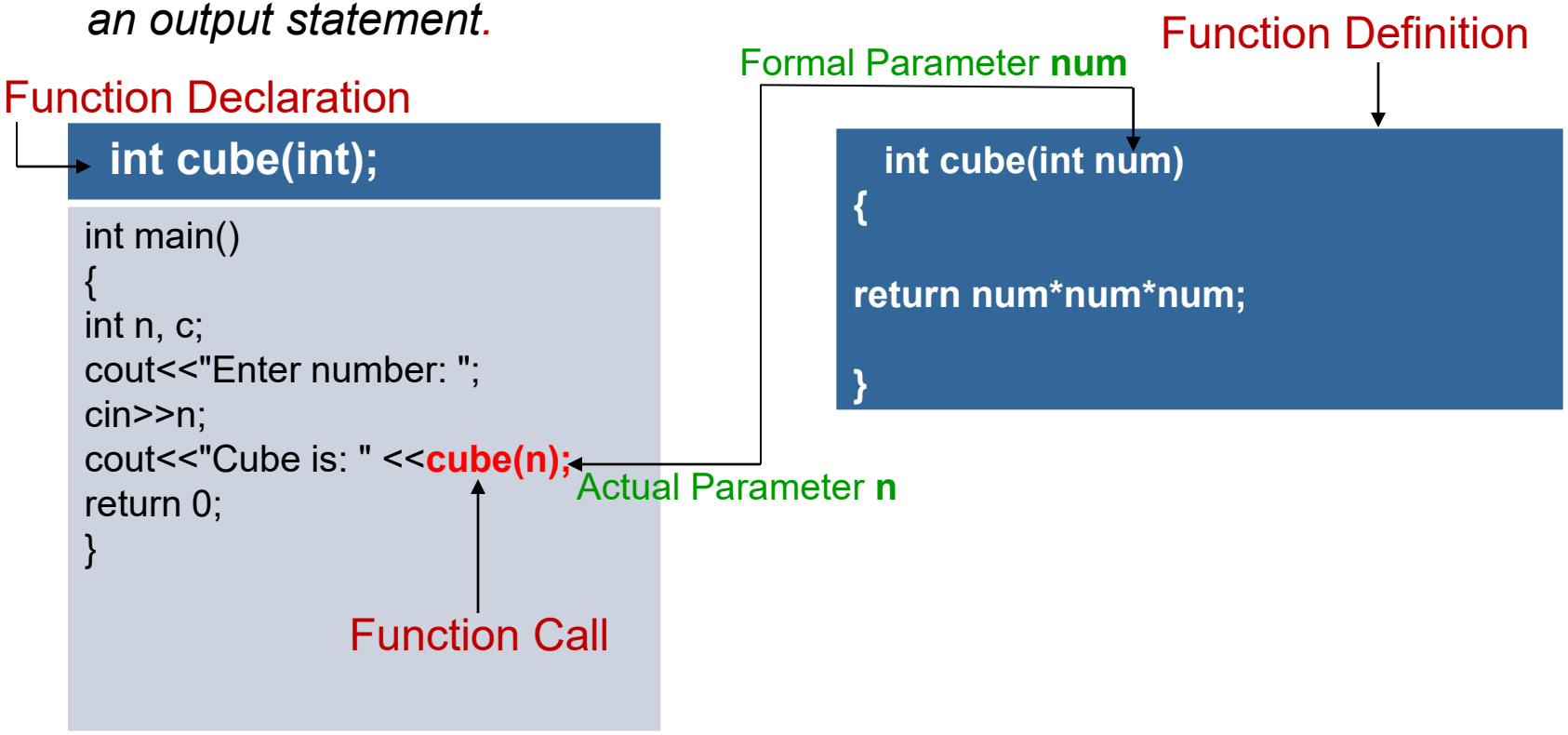
```
int sqr(int n)
{
        return n*n;
}

int cube(int n)
{
        return n*n*n;
}
```

Output



4.2 Returning Value from Function (continued…)

3. **Output Statement:** *The calling function can use the returned value directly in an output statement.*



Returned Value used in Output Statement

4.2 Returning Value from Function (continued…)

Example: A program that inputs two integers. It passes first integer to a function that calculates and returns its square. It passes second integer to another function that calculates and returns its cube. The **main()** function adds both returned values and displays the result.

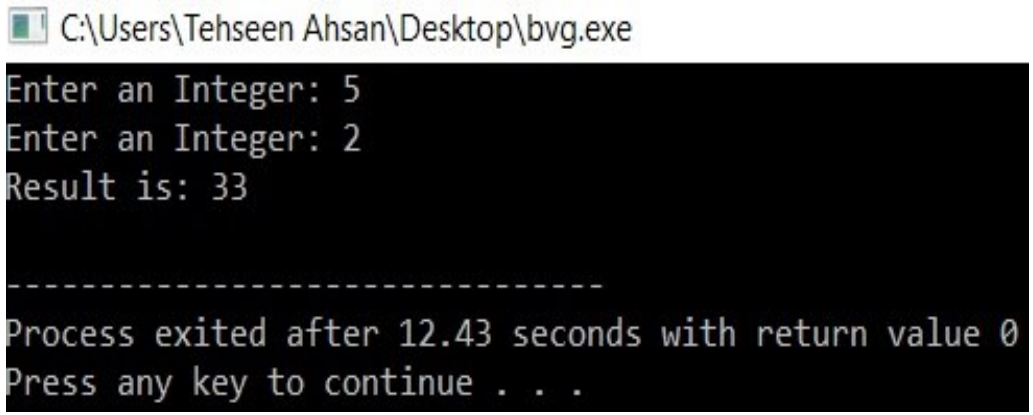
```
#include<iostream>
using namespace std;

int sqr(int n);
int cube(int n);

int main()
{
int a, b;
cout<<"Enter an Integer: ";
cin>>a;
cout<<"Enter an Integer: ";
cin>>b;
cout<<"Result is: "<< sqr(a) + cube(b);<<endl;
return 0;
}
```

Next slide

4.2 Returning Value from Function (continued…)

```
int sqr(int n)
{
        return n*n;
}

int cube(int n)
{
        return n*n*n;
}
```

Output



5. Local Variable

A variable declared inside a function is known as **local variable**. Local variables are also called **automatic variables**. The syntax of declaring a local variable is as follow:

auto data_type variable name;

auto: It is a keyword that indicates that the variable is automatic.

data_type: It indicates the data type of variable.

The use of keyword **auto** *is optional.*

5.1 Scope of Local Variable

- *The area where a variable can be accessed is known as* **scope of variable**.
- *Local variable can be used only in the function in which it is declared*.
- If a statement accesses a local variable that is not in scope, the computer will generate a syntax error.

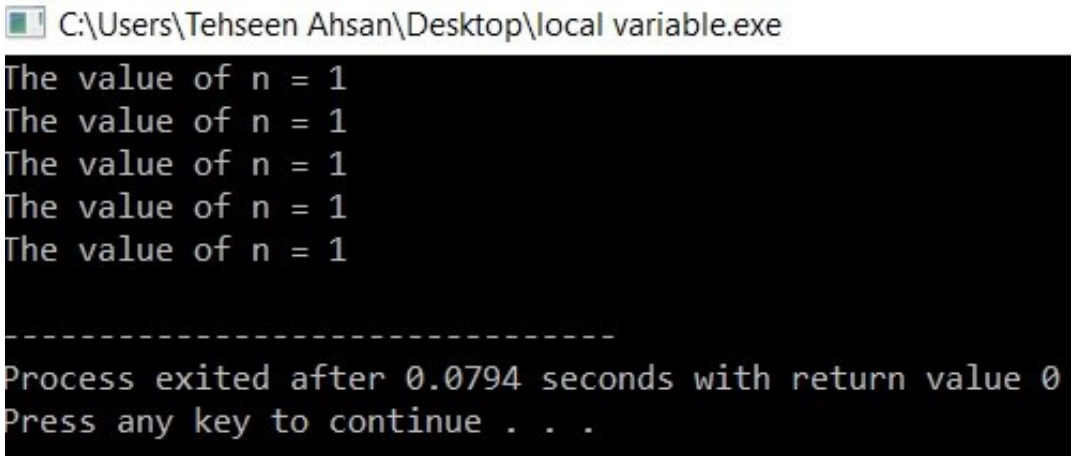
5.2 Lifetime of Local Variable

- *The time period for which a variable exists in the memory is known as the* **lifetime of variable**. Different types of variables have different lifetimes.
- *The lifetime of local variable starts when the control enters the function in which it is declared. Local variable is automatically destroyed when the control exits from the function and its lifetime ends*.
- *When the lifetime of a local variable ends, the value stored in this variable also becomes inaccessible*.

5.3 Example

```
#include<iostream>
using namespace std;

void fun();

int main()
{
int n;
for(int i=1;i<=5;i++)
{
        fun();
}
return 0;
}

void fun()
{
        int n = 0;
        n++;
        cout<<"The value of n = "<<n<<endl;
        }
```

Output



5.3 Example (continued…)

Working of a program:

The program in previous slide declares a function **fun()**. The function declares a local variable **n** and initializes it to 0. The main () function calls it five time using **for** loop. The control moves to the function in each function call. The local variable **n** is created in the memory and is initialized with 0. Finally, the value of **n** is displayed on screen. The variable **n** is destroyed from the memory when the control exits the function.

6. Global Variable

A variable declared outside any function is known as **global variable**. *Global variables can be used by all functions in the program*. *The values of these variables are shared among different functions*. If one function changes the value of a global variable, this change is also available to other functions.

6.1 Scope of Global Variable

- Global variable can be used by all functions in the program.
- It means that these variables are globally accessed from any part of the program.
- *Normally, global variables are declared before main function*.

6.2 Lifetime of Global Variable

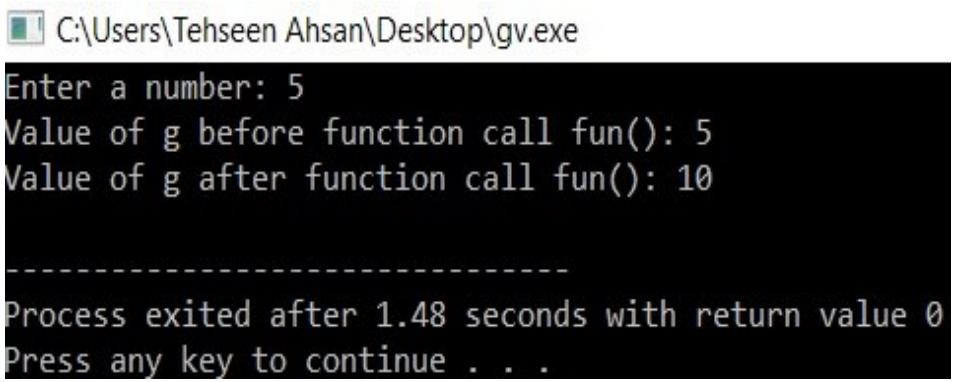
- *Global variables exist in the memory as long as the program is running*.
- *These variables are destroyed from the memory when the program terminates*.
- These variables occupy memory longer than local variables. So, global variables should be used only when these are very necessary.

6.3 Example

Write a program that inputs a number in a global variable. The program calls a function that multiplies the value of global variable by 2. The main function then displays the value of global variable.

```
#include<iostream>
using namespace std;
int g;
void fun();
int main()
{
cout<<"Enter a number: ";
cin>>g;
cout<<"Value of g before function call fun(): "<<g<<endl;
fun();
cout<<"Value of g after function call fun(): "<<g<<endl;
return 0;
}
void fun()
{
g = g*2;
}
```

Output



6.3 Example (continued…)

Working of a program:

The program in previous slide declares a global variable **g**. The variable is accessible to all functions in the program. The function **fun()** doubles the value of **g** when **main()** function calls it. Finally, **main()** function displays the increased value of **g**.

7. Difference between Local and Global Variable

Local Variable	Global Variable
1. Local variables are declared within a function.	1. Global variables are declared outside any function.
2. Local variables can be used only in the function in which they are declared.	2. Global variables can be used in all functions.
3. Local variables are created when the control enters the function in which they are declared.	3. Global variables are created when the program starts execution.
4. Local variables are destroyed when the control leaves the function.	4. Global variables are destroyed when the program terminates.
5. Local variables are used when the values are to be used within in a function.	5. Global variables are used when values are to be shared among different functions.

8. Static Variable, its Scope and its Lifetime

- *A local variable declared with keyword **static** is called* **static variable**.
- *The keyword* **static** *is used to increase the lifetime of local variable*.
- The **scope of static variable** is similar to the scope of local variable.
- Static variable can be used only in the function in which it is declared.
- *The* **lifetime of static variable** *is similar to global variable*. Static variables exist in the memory as long as the program is running.

8. Static Variable, its Scope and its Lifetime (continued…)

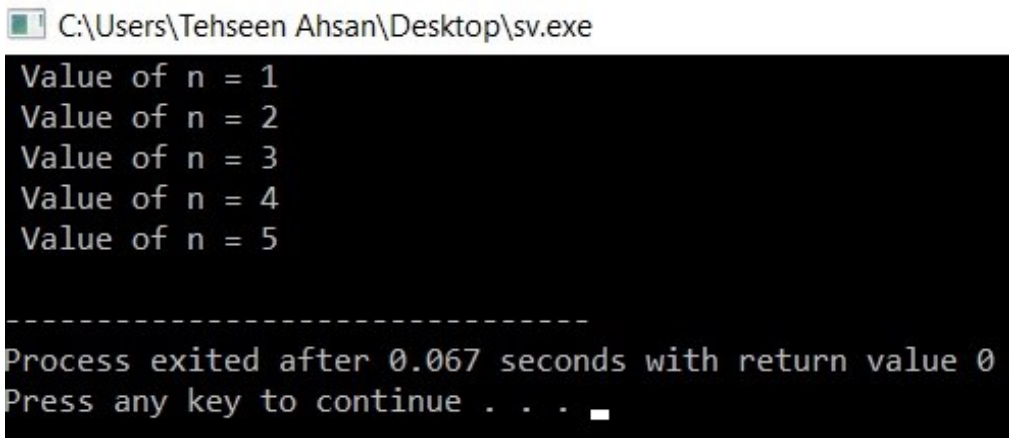
- These variables are destroyed from the memory when the program terminates.
- *Static variables are created in the memory when the function is called for the first time*. *When the control exits from the function, these variables are not destroyed unlike local variables*. *Moreover, the initialization statement for static variable is also executed only when the function is executed for the first time*.
- *A function that contains static variables runs faster than the function with local variables. This is because local variables are created each time the function is called but static variables are created only once*.

8.1 Example

*Write a program that calls a function for five times using **for** loop. The function uses a **static variable** initialized to 0. Each time the function is called, the value of the static variable is incremented by 1 and is displayed on the screen.*

```
#include<iostream>
using namespace std;
void fun();
int main()
{
int i;
for (i=1;i<=5;i++)
{
        fun();
}
return 0;
}
void fun()
{
static int n = 0;
n++;
cout<<" Value of n = "<<n<<endl;
}
```

Output



Try to run this code by inserting *int n = 0;* for static int n = 0; in the *function definition* and analyse the output.

8.1 Example (continued…)

Working of a program:

The program in previous slide declares a function **fun()**. The function declares a static variable **n** and initializes it to 0. The **main()** function calls it 5 times using **for** loop. The control moves to the function in each function call. The local variable **n** is created and initialized with 0 only in first function call. The last statement displays the value of **n** on the screen. The variable **n** is not destroyed from the memory when the control exists the function. That is why, the value of **n** is different in each function call.

9. Register Variables

A variable declared with **register** keyword is known as **register variable**. *The value of register variable is stored in registers instead of RAM*. *Registers are faster than RAM so the values stored in register can be accessed faster than the values stored in RAM*.

register data_type variable name;

register: It is a keyword that indicates that it is register variable.

data_type: It indicates the data type of variable.

Example

The following line declares a register variable.

register int n;

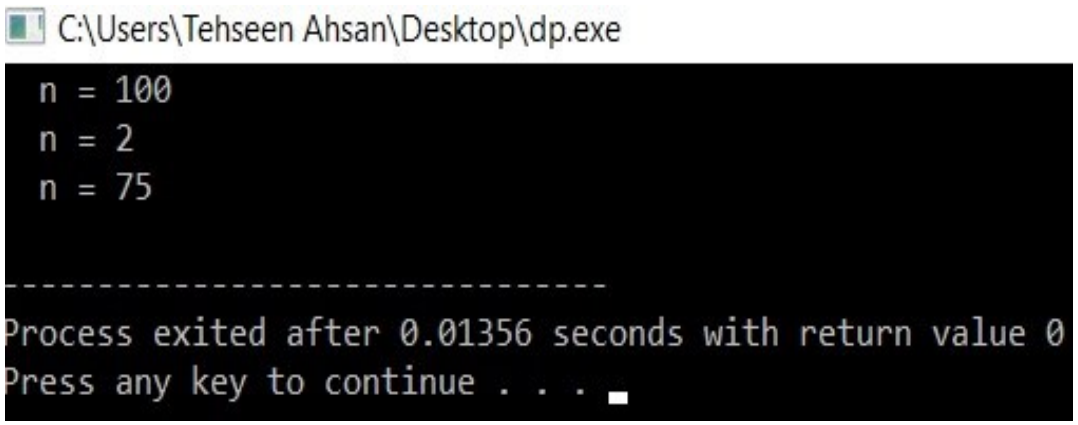
10. Default Parameters

- *The parameters that are initialized during function declaration are known as* **default parameters**.
- *The use of default parameters allows the user to call a function without giving the required parameters. The initialized values of default parameters are used if the user does not specify any value for the parameters in function call. The default values can be constants, global variables or function calls*.
- *The user can also specify the values in function call even if the default parameters are initialized. The values of default parameters are not used if the user specifies the values in function call. The default values are used only if the values in function call are not specified*.

10.1 Example no 1

```
#include<iostream>
using namespace std;
void Test (int n=100);
int main()
{
        Test(); // Test(100)
        Test(2);
        Test(75);
        return 0;
}
void Test(int n)
{
        cout<<"n = "<<n<<endl;
}
```

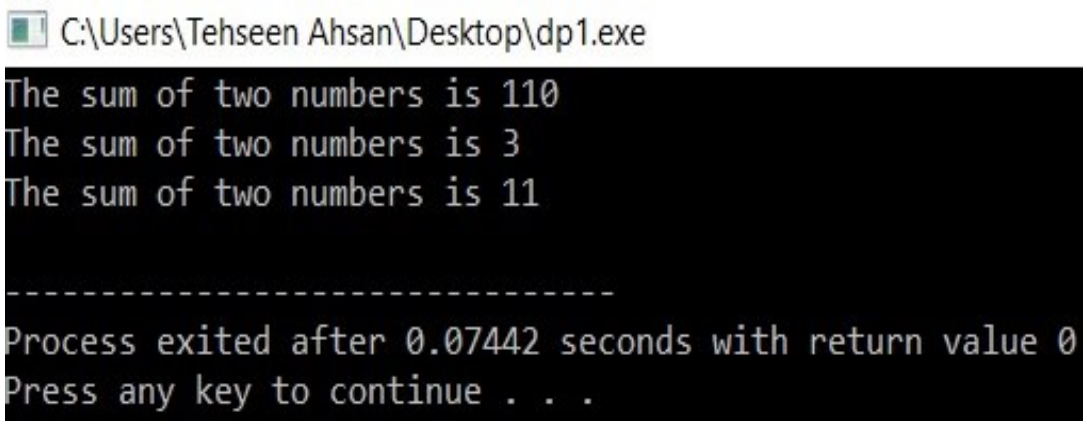
Output



10.1 Example no 2

```
#include<iostream>
using namespace std;
void add (int var1 =100, int var2 =10);
int main()
{
        add(); //add(100,10)
        add(1,2);
        add(1);  //add(1, 10)
        return 0;
}
void add (int var1, int var2)
{
        int var3;
        var3 = var1 + var2;
        cout<<"The sum of two numbers is "<<var3<<endl;
}
```

Output



11. Inline Functions

- The function declared with keyword **inline** is known as **inline function**. *The inline function is declared before main function*. Only very small functions are declared as inline functions.
- *When a program is compiled, the compiler generates **special code** at each function call to move the control to the called function. It also generates **special code** at the end of function definition to move the control back to the calling function*. *The shifting of control from calling function to the called function and back to the calling function takes time during program execution. This time can be eliminated by using **inline functions***.

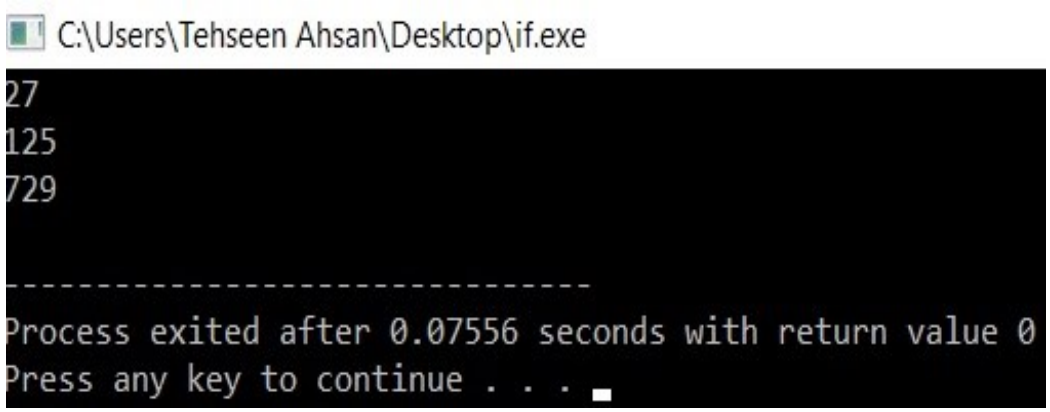
11. Inline Functions (continued…)

- The code of function body is inserted at each function call if the function is declared as inline function. It allows the program to execute the function statements without shifting the control from calling function. It saves time and increases program's efficiency. ***(ADVANTAGE)***
- The code of function body is repeated at each function call. It takes more memory than normal function call. That is why the inline functions must only be used if the function body contains a small number of statements. ***(DISADVANTAGE)***

11.1 Example

```
#include<iostream>
using namespace std;
inline int cube(int);
int main()
{
        cout<<cube(3)<<endl;
        cout<<cube(5)<<endl;
        cout<<cube(9)<<endl;
        return 0;
}
inline int cube(int n)
{
        return n*n*n;
}
```

Output



12. Function Overloading

- *The process of declaring multiple functions with same name but different parameters is known as* **function overloading**.
- *The functions with same name must differ in one of the following ways*:
  - *Number of parameters*
  - *Type of parameter*
  - *Sequence of parameters*
- Function overloading allows the programmer to assign same name to all functions with similar tasks. It helps the user in using different functions easily.
- The user doesn't need to remember many names for similar types of tasks.

12. Function Overloading (continued…)

- For example, the programmer can declare the following functions to calculate average:
  - **float Average(int, int) ;**
  - **float Average(float, float) ;**
  - **float Average(int, float) ;**
  - **float Average(float, int) ;**
- The above four functions are used to calculate average but these functions accept different types of parameters. The user can use the same name **Average** and pass different types of parameters. The user must think that one function is working in different ways.
- When the user calls a function, the compiler checks the parameters and their types. The corresponding function is executed according to the types, number and sequence of the parameters in function call.

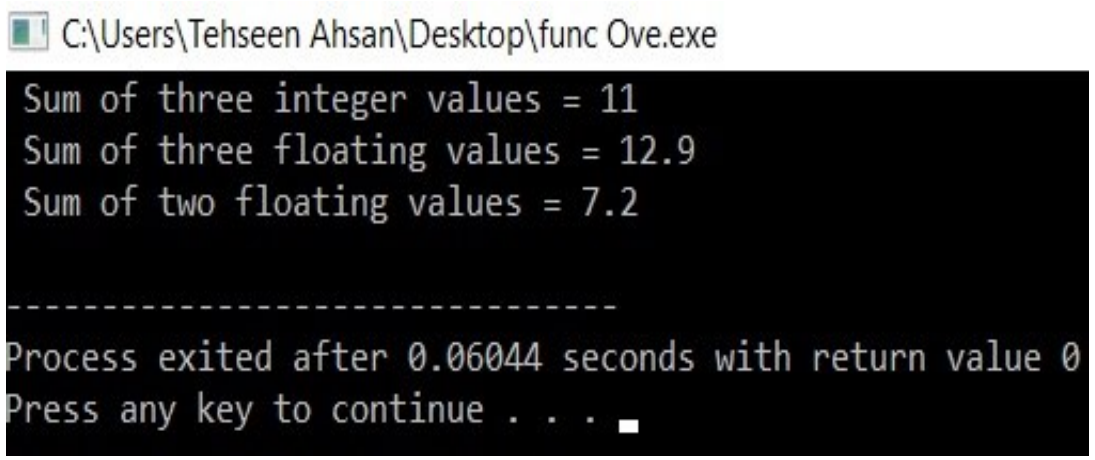
12.1 Example

```
#include<iostream>
using namespace std;
int sum(int, int, int);
double sum(double, double, double);
float sum(float, float);


int main()
{
cout<<" Sum of three integer values = "<<sum(2,4,5)<<endl;
cout<<" Sum of three double values = "<<sum(2.6,4.7,5.6)<<endl;
cout<<" Sum of two floating values = "<<sum(2.5,4.7)<<endl;
return 0;
}
```

12.1 Example (continued…)

```
int sum(int x, int y, int z)
{
        return x+y+z;
}
double sum(double x, double y, double z)
{
        return x+y+z;
}
float sum(float x, float y)
{
        return x+y;
}
```

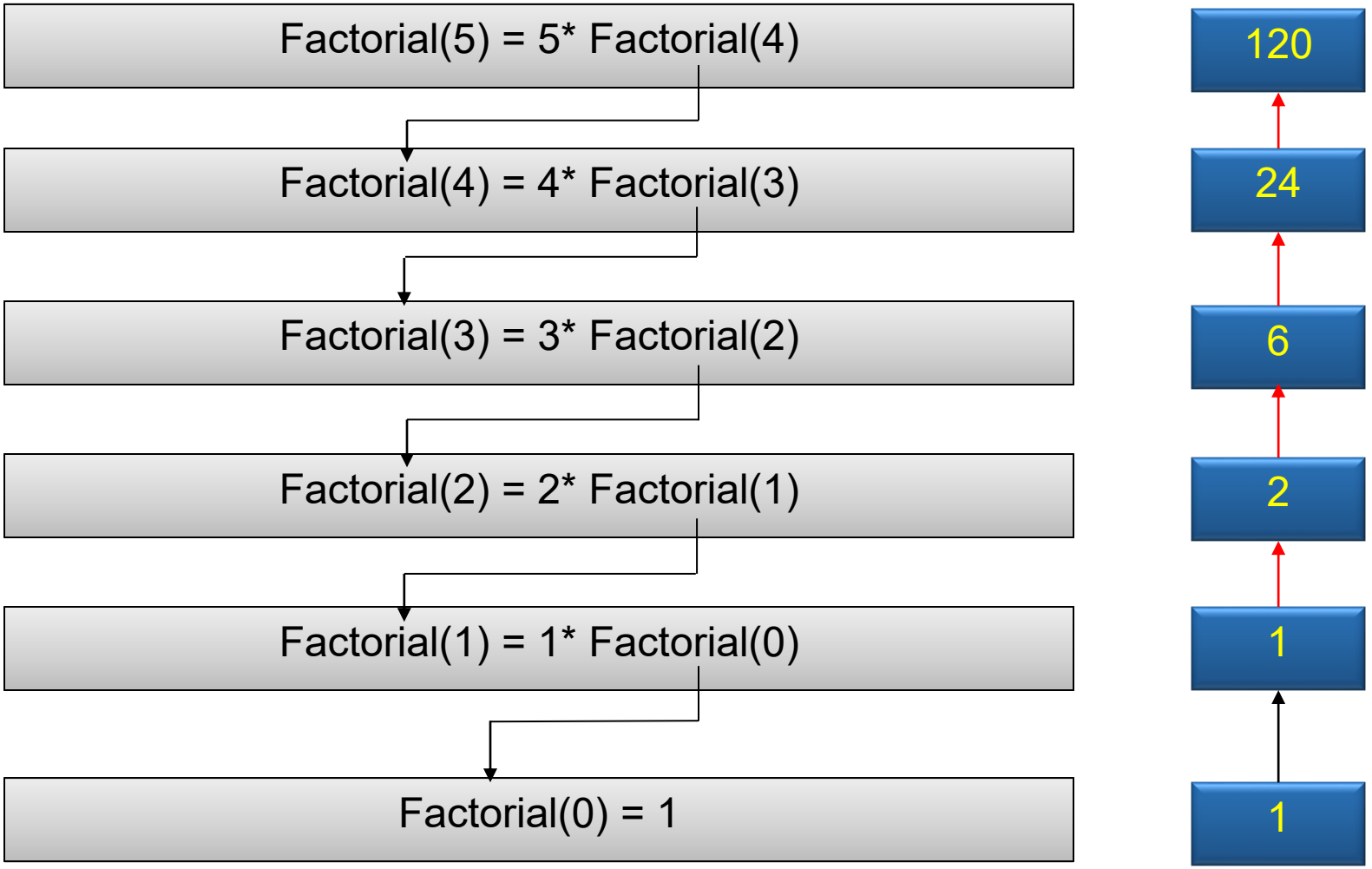
Output



13. Recursion

- *The programming technique in which a function calls itself is known as* **recursion**.
- *A function that calls itself is called* **recursive function**. The recursion is a powerful technique.
- *For example the calculation of factorial of a number is simple example of recursive technique*.
- *Suppose the factorial of 5 is to be calculated. The recursive solution of this problem can be shown in next slide*.
- The figure next slide shows that factorial can be calculated by decomposing the problem. The factorial of 5 can be calculated by multiply 5 with the factorial of 4. The factorial of 4 is multiplied by 4 and so on.

13. Recursion (continued…)



Recursive calculation of factorial

13. Recursion (continued…)

- The procedure in previous slide can be implemented as follows:

```
int Factorial (unsigned int n)
{
if(n==0)
      return 1;
else
      return n*Factorial(n-1);
}
```

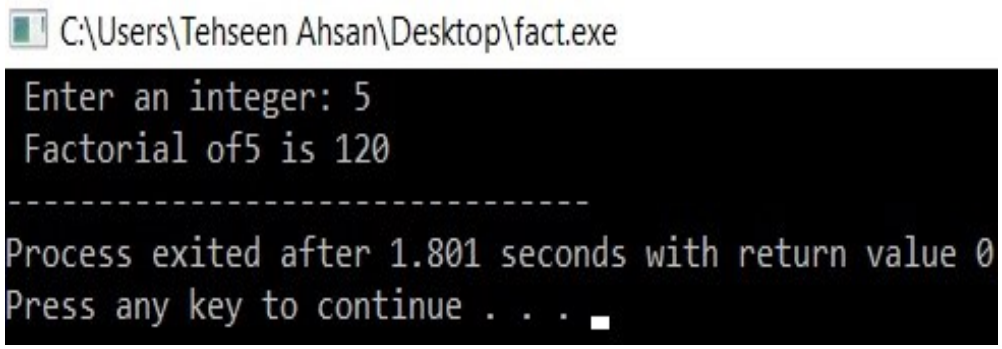
Loop will exit while returning value 1

13. Recursion (continued…)

Example: *Write a program that inputs a number from user and calculates its factorial recursively.*

```
#include<iostream>
using namespace std;
long int fact(int);
int main()
{
        int num;
        cout<<" Enter an integer: ";
        cin>>num;
        cout<<" Factorial of"<< num<<" is "<<fact(num);
        return 0;
}
long int fact (int n)
{
if(n==0)
        return 1;
else
        return n*fact(n-1);
}
```

Output



References: Textbooks

1. *"C++ Programming: From Problem Analysis to Program Design",6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *"Object Oriented Programming in C++," 4/ed by Robert Lafore (2001) .*
3. *"C++ How to Program," 5/ed by Deitel & Deitel (2005).*
4. *"Program with C++ Object Oriented Programming Aikman Series", by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*